

Sistema de Detección y Segmentación de Objetos en Tiempo Real mediante Ajuste Fino de YOLOv8

Trabajo Práctico Final — Deep Learning

Federico Morán

Maestría en Inteligencia Artificial

Julio de 2026

- 1 Problema y alcance
- 2 Datos, modelo y entrenamiento
- 3 Evaluación y resultados
- 4 Sistema en tiempo real
- 5 Conclusiones

Hilo conductor

Ajustar un modelo de segmentación de instancias y llevarlo a tiempo real sobre una CPU... **y descubrir que medir bien el resultado es tan difícil como obtenerlo.**

Objetivo

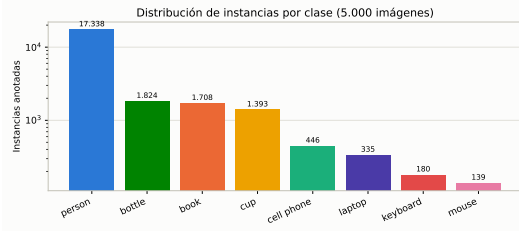
Un sistema completo de visión por computadora que **detecte y segmente objetos en tiempo real** sobre hardware de consumo: datos → ajuste fino → evaluación → inferencia en vivo.

¿Por qué el tiempo real? Robótica, asistencia y análisis de video exigen **precisión** y **latencia** a la vez, sobre todo sin aceleración dedicada.

Ocho clases

person cell phone cup bottle
laptop keyboard mouse book

Criterio: todas pueden **exhibirse físicamente** ante una cámara web en la demo: es un criterio de aplicación, no de conveniencia estadística. Las 72 categorías restantes de COCO pasan a ser **fondo**, lo que define con precisión el alcance del problema.



Desbalance pronunciado, heredado de la estadística natural de COCO: person concentra el 74,2 % de las instancias; mouse reúne 139.

Dos tareas, un modelo — y las anotaciones que lo permiten

Detección — localiza cada objeto con una *caja delimitadora* y le asigna una clase.

Segmentación de instancias — delimita, *píxel a píxel*, la silueta exacta de cada objeto individual.

Clave

YOLOv8-seg resuelve **ambas** tareas en una única pasada hacia adelante: la cabeza predice caja, clase y coeficientes de máscara simultáneamente.

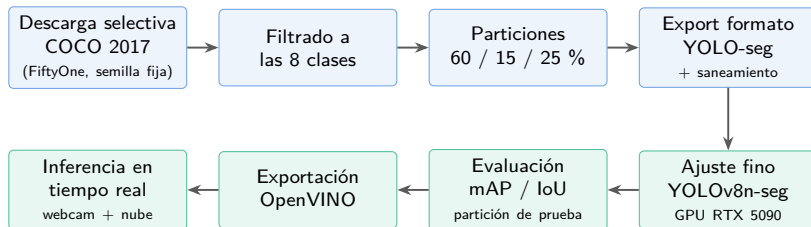
Por eso el conjunto de datos se anota a nivel de **instancia** (polígonos, no solo cajas): 5.000 imágenes y 23.363 instancias de COCO 2017.

Verificación: anotaciones reconstruidas desde los archivos YOLO exportados



Control de calidad: las anotaciones se reconstruyen *desde los archivos YOLO exportados* y se superponen sobre las imágenes. Valida la cadena completa: formato, normalización de coordenadas y asignación de índices de clase.

Pipeline de extremo a extremo



Particiones disjuntas a nivel de imagen

3.000 entrenamiento / **750** validación / **1.250** prueba.

La prueba queda reservada: no interviene en ninguna decisión.

Reproducibilidad y controles

Semillas fijas en todo el pipeline. Un **control de integridad sobre los archivos exportados** —lo que el entrenador realmente lee— verifica que las particiones sean disjuntas y detiene la corrida si no lo son.

Modelo y estrategia: por qué congelamos el backbone

- **YOLOv8n-seg** — una etapa, libre de anclas: *backbone* CSPDarknet → *neck* PAN-FPN → cabeza desacoplada (caja, clase y *coeficientes de máscara*) + rama de *prototipos*.
La máscara es una combinación lineal de prototipos ⇒ costo de segmentación casi constante por imagen.
- **Variante *nano*** (3,26 M de parámetros) — el tiempo real debe darse **sobre CPU**, sin GPU dedicada.
- **Se parte de los pesos preentrenados en COCO**, pero el vocabulario baja de 80 a 8 clases:
 - ▶ **Se hereda** — backbone y neck completos: las representaciones visuales aprendidas sobre 118.000 imágenes.
 - ▶ **Se reinicializa** — 36 tensores (~372.000 parámetros): **toda la rama de clasificación de la cabeza**, cuyas dimensiones dependen de `nc`.
- El propio entrenador lo declara: `Overriding nc=80 with nc=8 Transferred 381/417 items`

Decisión: congelar el backbone (`freeze=10`)

El clasificador arranca **desde cero** y solo hay 3.000 imágenes: 40× menos que el preentrenamiento. Reentrenar la red entera arriesga **destruir representaciones buenas** para reaprenderlas con muy pocos datos ⇒ se congela el backbone y todo el presupuesto de optimización va al *neck* y a la cabeza. **La ablación de la próxima sección confirma que era la decisión correcta.**

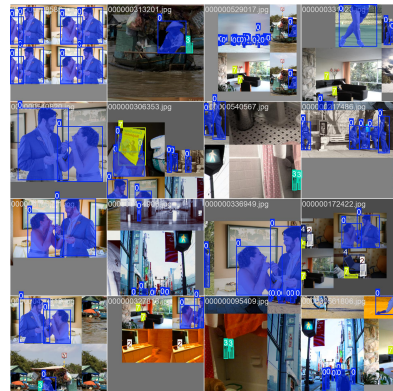
Entrenamiento: aumento de datos y configuración

Transformación	Valor	Justificación
<i>Mosaic</i>	1,0	Contextos y escalas atípicos
Volteo horizontal	$p=0,5$	Sin quiralidad relevante
Traslación	$\pm 10\%$	Encuadres imperfectos
Escala	$\pm 50\%$	Distancia objeto-cámara
HSV	0,015/0,7/0,4	Iluminación y color

Transforma de manera consistente las imágenes y sus polígonos.

Mosaic se desactiva en las últimas 10 épocas. **Se omiten** volteo vertical, rotaciones y *mixup*: generarían poses inverosímiles para el dominio. Entrada 640×640 por *letterbox*.

Base / estrategia	yolov8n-seg.pt, freeze=10
Épocas	150 (early stop en la 140)
Lote / optimizador	32 / AdamW, $lr_0 = 8,33 \times 10^{-4}$
Hardware / duración	RTX 5090 — 132 min



El punto de control se selecciona por la métrica de aptitud sobre **validación**. Curvas de aprendizaje en los anexos: sin sobreajuste.

Resultados sobre la partición de prueba reservada

Modelo ajustado	Cajas	Máscaras
Precisión media	0,577	0,559
Recall medio	0,397	0,383
mAP@50	0,440	0,420
mAP@50-95	0,292	0,246

mAP@50-95 —promedio sobre umbrales de IoU de 0,5 a 0,95— es la métrica principal: premia la **precisión de la localización**. Se calcula por separado para cajas y máscaras sobre 1.250 imágenes y 7.117 instancias.

El recall es el cuello de botella

La precisión (0,577) supera con holgura al recall (0,397): el modelo acierta cuando detecta, pero **omite** instancias. Las máscaras puntúan por debajo de las cajas: el IoU a nivel de píxel es más estricto.

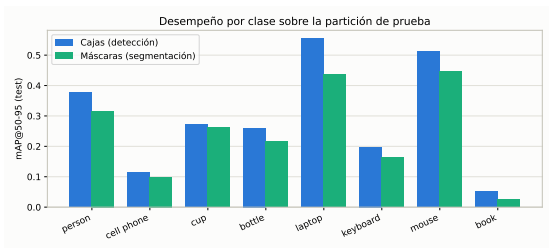
Ablación: el congelamiento estaba justificado

Configuración	Cajas	Másc.
Ajuste fino <i>completo</i> , 60 ép.	0,250	0,216
Backbone congelado , 150 ép.	0,292	0,246

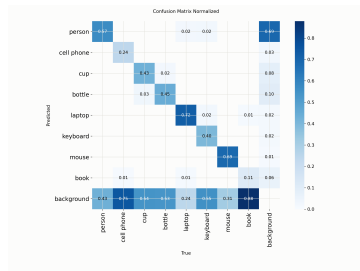
Ambas corridas parten de la **misma cabeza reinicializada**. Reentrenar la red entera con 3.000 imágenes *degrada* el modelo: se pierden representaciones que COCO tardó 118.000 imágenes en construir.

Lección: cuando el vocabulario de destino es un **subconjunto** del de preentrenamiento, un ajuste fino ingenuo puede *empeorar* el modelo. Congelar el backbone vuelve **segura** la transferencia.

¿Dónde falla el modelo?



Mejores: laptop (0,554) y mouse (0,512), objetos rígidos y poco ocluidos. Peor: book, apilado en estanterías, con límites ambiguos y anotación inconsistente.



El modelo **prácticamente no confunde clases entre sí** (máx. 0,03). El modo de error dominante es la **omisión**: objetos reales clasificados como fondo (book 88 %, cell phone 75 %).

Lectura conjunta

Con un vocabulario reducido la *clasificación* es sencilla; lo difícil es *detectar* instancias pequeñas u ocluidas. El margen de mejora está en el **recall**, no en la discriminación entre clases.

El problema: no se puede comparar sobre nuestra partición de prueba

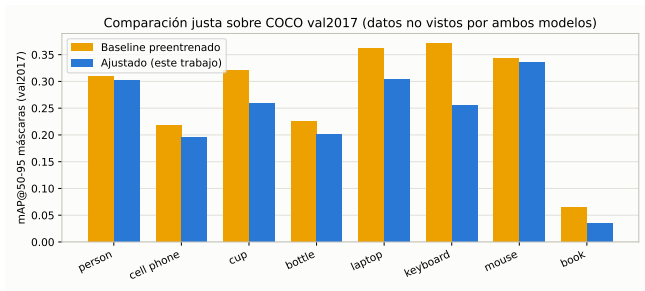
El baseline (`yolo_v8n-seg.pt`) se entrenó sobre COCO **train2017**, el mismo conjunto del que muestreamos nuestras 5.000 imágenes. **Ya vio, durante su preentrenamiento, las imágenes que nosotros usamos como prueba**, mientras que nuestro modelo se evalúa sobre datos genuinamente reservados. Medirlos ahí no compararía modelos: compararía memoria contra generalización.

La solución: un tercer conjunto que ninguno de los dos vio

Se replicó la receta de datos del Notebook 01 sobre **COCO val2017** —el conjunto de validación oficial, **disjunto de train2017**, que el preentrenado nunca usó para ajustar sus pesos— y se obtuvieron **1.250 imágenes nuevas**. Ambos modelos se evalúan sobre *esas mismas* imágenes, cada uno con su vocabulario nativo (80 y 8 clases); del baseline se extraen las AP de las 8 clases de interés.

⇒ Ninguno de los dos vio los datos: **la comparación es limpia**.

Resultado de la comparación imparcial (COCO val2017)



mAP@50-95	Preentr.	Ajustado
Máscaras	0,277	0,236
Cajas	0,329	0,275

Sobre datos que ninguno vio, el modelo ajustado queda a **0,041** del preentrenado, con **paridad** en person (−0,008) y mouse (−0,007). La mayor diferencia persiste en keyboard (−0,115), la clase con menos ejemplos de entrenamiento (116).

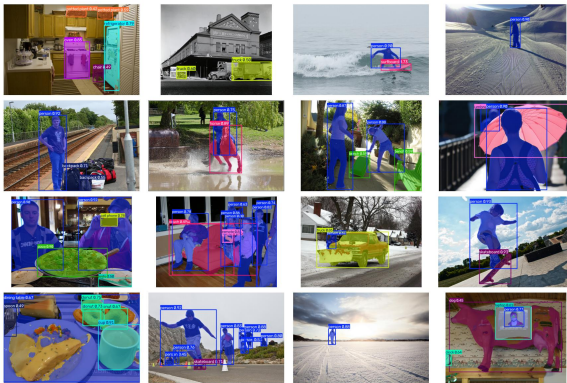
Por qué el preentrenado es un baseline tan fuerte — y qué significa igualarlo

Las 8 clases son un **subconjunto** de su vocabulario: las aprendió sobre 118.000 imágenes. Ajustar con 3.000 del mismo dominio no agrega información nueva, así que lo esperable es **igualarlo, no superarlo en mAP**. Entonces, ¿para qué sirvió el ajuste fino?

Entonces, ¿qué aporta el ajuste fino? Especialización

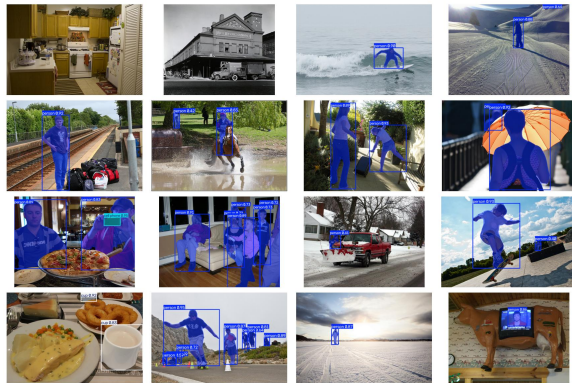
Baseline (80 clases)

Modelo preentrenado de referencia (80 clases) — COCO val2017



Ajustado (8 clases)

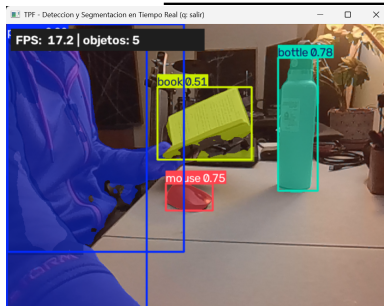
Modelo ajustado (8 clases, este trabajo) — COCO val2017



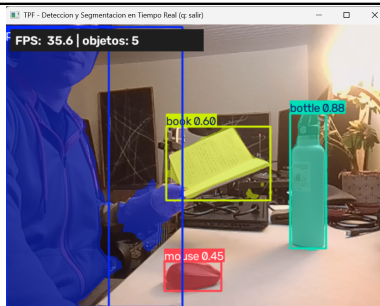
- El baseline dispersa detecciones fuera de alcance (refrigerator, surfboard, horse, umbrella, pizza, cow...).
- El ajustado se concentra **exclusivamente** en las 8 clases: suprime por completo los falsos positivos de vocabulario. *Esta* es su ventaja práctica, invisible para el mAP por clase.

Tiempo real sobre CPU: el efecto de OpenVINO

Plataforma	Formato	Latencia	FPS
Servidor, GPU RTX 5090	PyTorch (.pt)	5,1 ms/imagen	~198
PC personal, CPU Core Ultra 5	PyTorch (.pt)	~59 ms/cuadro	17
PC personal, CPU Core Ultra 5	OpenVINO	~29 ms/cuadro	35



PyTorch: 17,2 FPS



OpenVINO: 35,6 FPS

OpenVINO **duplica** la tasa de cuadros y supera el umbral de tiempo real (24–30 FPS) sin aceleración dedicada.

`src/demo_webcam.py`: captura con OpenCV, inferencia cuadro a cuadro y superposición de cajas, máscaras, etiquetas y FPS (media móvil exponencial).

La PC de demostración **no tiene GPU con CUDA** $\Rightarrow$ se exporta el modelo a **OpenVINO**, la representación intermedia de Intel especializada para CPU.

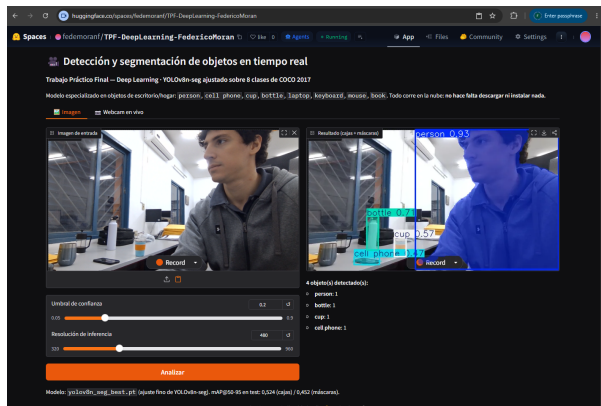
Despliegue en la nube: Hugging Face Spaces

La aplicación se desplegó en un **servidor público** con interfaz Gradio: quien la evalúe solo abre un enlace, sin descargar ni instalar nada.

Dos modos de uso

- **Imagen** — sube una fotografía; devuelve cajas, máscaras y recuento de objetos por clase.
- **Webcam en vivo** — inferencia cuadro a cuadro sobre la cámara del navegador, con umbral de confianza y resolución expuestos.

Los pesos viajan dentro del repositorio del Space: el modelo se ejecuta íntegramente en la nube.



Demostración en vivo

Detección y segmentación de instancias sobre cámara web

CPU Intel Core Ultra 5 226V — sin GPU dedicada

- ❶ **La evaluación rigurosa es tan determinante como el modelo.** Se corrigieron dos sesgos optimistas: la fuga en el propio *split* y el preentrenamiento del baseline sobre el mismo pozo de datos del que salió nuestro test. Bajo una comparación imparcial en val2017 el modelo ajustado es **competitivo** con el preentrenado.
- ❷ **Con un vocabulario que es subconjunto del preentrenamiento, el aporte del ajuste fino es la especialización, no el mAP:** elimina las detecciones espurias de las 72 categorías fuera de alcance.
- ❸ **El error dominante es la omisión, no la confusión entre clases:** el desempeño por clase lo gobiernan las características visuales y la calidad de anotación de cada categoría.
- ❹ **La segmentación de instancias en tiempo real es viable sobre una CPU de consumo:** 35 FPS sostenidos con OpenVINO, el doble que el formato PyTorch original.
- ❺ **El pipeline es reproducible de extremo a extremo,** con semillas fijas y controles de integridad sobre los archivos exportados.

Limitaciones

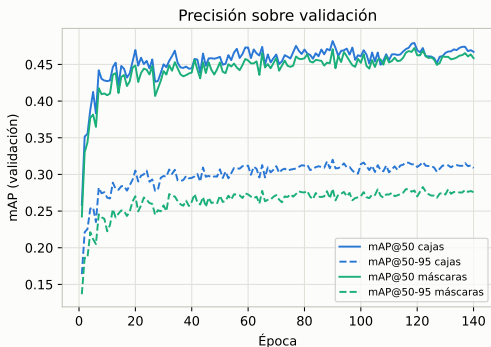
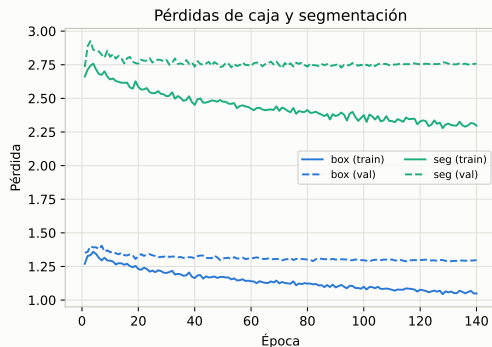
- **Desbalance de clases:** person concentra el 74,2 % de las instancias; mouse y keyboard tienen menos de 150 ejemplos de entrenamiento. No se corrigió artificialmente: es la estadística del dominio de despliegue.
- **Calidad de la anotación de referencia:** book tiene límites ambiguos en COCO, lo que impone un techo a su desempeño medible.
- **Escala del conjunto:** 3.000 imágenes son dos órdenes de magnitud menos que el preentrenamiento; por eso congelar el backbone es determinante.

Trabajo futuro

- **Ampliar el conjunto de datos**, en especial las clases raras.
 - **Atacar el recall** de instancias pequeñas: inferencia en mosaico (*tiling*) o mayor resolución de entrada.
 - **Variantes de mayor capacidad** (YOLOv8s-seg), midiendo el intercambio precisión-latencia sobre la CPU de despliegue.
 - **Cuantización INT8** con OpenVINO para elevar aún más los FPS.
- ⇒ La única vía para **superar** genuinamente al baseline es entrenar con datos del **dominio real** (imágenes de webcam), no con más COCO.

¿Preguntas?

Diapositivas de respaldo



- Las pérdidas de caja y segmentación decrecen de forma sostenida, sin divergencia: no hay sobreajuste.
- El mAP de validación crece rápido y se estabiliza en una meseta (mAP@50-95 de máscaras hasta 0,283), coherente con un modelo de *backbone* congelado.
- La pérdida de validación se ubica **por encima** de la de entrenamiento, como corresponde a la brecha de generalización sobre una partición reservada.

La comparación que *no* se presentó, y por qué

Evaluar al preentrenado sobre *nuestra* partición de prueba (muestreada de COCO train2017, con la que él fue entrenado) lo favorece: mide, en parte, su memoria sobre sus propios datos de entrenamiento. Por eso la comparación del cuerpo de la presentación se hace únicamente sobre COCO val2017.

mAP@50-95	Preentrenado		Ajustado	
	Cajas	Máscaras	Cajas	Máscaras
Nuestro test (de train2017) — <i>sesgado</i>	0,403	0,342	0,292	0,246
COCO val2017 — imparcial	0,329	0,277	0,275	0,236

La magnitud del sesgo

Al pasar a datos no vistos el baseline **cae** de 0,342 a 0,277 en máscaras, mientras que el ajustado apenas varía (0,246 → 0,236): para él ambas particiones son igualmente novedosas. Cerca de **dos tercios de la brecha aparente** eran la ventaja de fuga del baseline, no un defecto del modelo ajustado.

Un riesgo real, detectado y corregido

FiftyOne etiqueta cada muestra descargada con su *split de origen* (train, por venir de COCO train2017). Esa etiqueta **colisiona** con la de la partición propia: `match_tags('train')` devuelve las 5.000 imágenes, no las 3.000 previstas, y el modelo entrena sobre validación y prueba.

Tres correcciones encadenadas

- 1 `untag_samples` elimina las etiquetas de split de origen *antes* de particionar.
- 2 `rmtree` del directorio de destino antes de exportar: el exportador de FiftyOne **fusiona** por defecto con lo que ya haya en disco.
- 3 Un `assert` sobre los **archivos exportados** verifica que las particiones sean disjuntas y midan 3.000 / 750 / 1.250.

Verificación sobre los archivos, no sobre las etiquetas

El control definitivo se hace sobre lo que el entrenador *realmente lee del disco*. Síntoma que delató el problema: el log de Ultralytics decía `train: Scanning...5000 images`.

IoU — solapamiento entre la predicción P y la anotación G :

$$IoU(P, G) = \frac{|P \cap G|}{|P \cup G|}$$

Se aplica **tanto a cajas como a máscaras**. Una predicción es verdadero positivo si su IoU con una anotación de la misma clase supera un umbral.

AP — área bajo la curva precisión–recall de una clase.

mAP — promedio de las AP sobre las clases.

Las dos variantes estándar

mAP@50 — umbral de IoU de 0,5. Criterio laxo: mide si el objeto fue encontrado.

mAP@50-95 — promedio sobre umbrales de 0,5 a 0,95 en pasos de 0,05. Criterio estricto: premia la **precisión de la localización**. Es la métrica principal de este trabajo.

Todo se calcula por separado para **cajas** (detección) y **máscaras** (segmentación), sobre la partición de prueba (1.250 imágenes, 7.117 instancias), reservada de principio a fin.

Parámetro	Valor
Modelo base	yolov8n-seg.pt (preentrenado en COCO)
Capas congeladas	freeze=10 (backbone); se entrenan neck y cabeza
Épocas	150 (early stopping, paciencia 50)
Resolución de entrada	640 × 640 (letterbox)
Tamaño de lote	32
Procesos de precarga	4
Optimizador	AdamW (selección automática), $lr_0 = 8,33 \times 10^{-4}$, momento 0,9
Decaimiento de pesos	5×10^{-4}
Precisión mixta (AMP)	Sí
Semilla	42 (modo determinista)
Hardware	GPU NVIDIA RTX 5090 (32 GB)
Duración	132 minutos (se detuvo en la época 140)

El lote y los procesos de precarga se fijaron explícitamente para acotar el consumo de memoria en un servidor compartido. El punto de control se selecciona por la **métrica de aptitud sobre validación**.

¿Por qué YOLOv8 y no una arquitectura de dos etapas (Mask R-CNN)?

El requisito de tiempo real sobre CPU descarta las arquitecturas de dos etapas. YOLOv8-seg resuelve detección y segmentación en una sola pasada, con costo de máscara casi constante gracias a los prototipos.

¿Por qué la variante nano y no small o medium?

Es la única que sostiene tiempo real sobre la CPU de despliegue. La variante *small* quedó identificada como trabajo futuro, con la latencia como criterio de decisión.

¿Por qué AdamW y esa tasa de aprendizaje?

Selección automática de Ultralytics (`optimizer='auto'`), que fija $lr_0 = 8,33 \times 10^{-4}$ y momento 0,9 en función del tamaño del conjunto y del lote.

¿Por qué no se corrigió el desbalance con pesos de clase o focal loss?

El desbalance es la estadística natural del dominio y se traslada al escenario de despliegue. Corregirlo artificialmente optimizaría una distribución que no es la de uso.

¿Cómo se garantiza que no hay fuga de datos entre train y test?

Con dos controles independientes: (i) `untag_samples` elimina las etiquetas de split de origen de FiftyOne antes de particionar; (ii) un `assert` verifica, sobre los *archivos exportados* —que es lo que el entrenador realmente lee—, que las tres particiones sean disjuntas y midan 3.000 / 750 / 1.250.

¿Por qué la pérdida de validación estuvo por debajo de la de entrenamiento en corridas previas?

Era un síntoma. Con particiones correctas la pérdida de validación queda *por encima*, como corresponde. En una corrida anterior, val estaba contenida en train.

¿El aumento de datos se aplica también a las máscaras?

Sí. Ultralytics transforma imágenes y polígonos de forma consistente; *mosaic* recompone las cuatro máscaras en el lienzo combinado.

¿Por qué `close_mosaic`?

Mosaic produce imágenes de estadística artificial; `close_mosaic=10` lo desactiva en las últimas 10 épocas para afinar sobre imágenes naturales. En esta corrida el *early stopping* (época 140) se adelantó a esa fase, sin perjuicio para la convergencia.

¿Por qué el baseline se evalúa con 80 clases y el modelo ajustado con 8?

Porque son los vocabularios nativos de cada modelo; de las AP del baseline se extraen las 8 clases de interés sobre las mismas imágenes. Y —clave— la comparación principal se hace sobre COCO **val2017**, no sobre nuestra partición de train2017: el preentrenado fue entrenado sobre train2017, así que evaluarlo ahí lo favorece. Sobre val2017, no vista por ninguno de los dos, la comparación es limpia.

¿La partición de prueba se usó alguna vez para tomar decisiones?

No. El punto de control se selecciona por el mAP@50-95 de máscaras sobre **validación**. La prueba se evalúa una sola vez, al final.

¿Los FPS dependen del modelo entrenado?

No. La latencia depende de la arquitectura y del formato de ejecución, no de los valores de los pesos. Las mediciones de 17 y 35 FPS son estables entre corridas.

¿Qué pasa si la cámara falla en la demo?

Hay un video de respaldo: `python src/demo_webcam.py -source respaldo.mp4`.

Requisito	Dónde se cumple
Detección y/o segmentación	YOLOv8n-seg ajustado: cajas + máscaras
Preprocesamiento y <i>data augmentation</i>	Notebooks 01 y 02
Métricas estándar (mAP, IoU)	Notebook 03: mAP@50 y mAP@50-95
Inferencia en tiempo real	src/demo_webcam.py (FPS en pantalla)
Pipeline completo	Notebooks 01→03 + src/
Modelo entrenado	models/yolov8n_seg_best.pt
Reporte técnico con código por paso	reporte/informe.pdf
Demo en la nube, solo un enlace	Hugging Face Space (Gradio)
Presentación oral con demo	Este documento + demo en vivo